```cpp
#include "SortablePair.h"

template<typename K, typename V>
SortablePair<K, V>::SortablePair( const K& aFirst, const V& aSecond ) noexcept
    : fFirst(aFirst), fSecond(aSecond)
{

}

template<typename K, typename V>
const K& SortablePair<K, V>::first() const noexcept
{
    return fFirst;
}

template<typename K, typename V>
const V& SortablePair<K, V>::second() const noexcept
{
    return fSecond;
}

template<typename K, typename V>
bool SortablePair<K, V>::operator==( const SortablePair& aOther ) const noexcept
{
    return fFirst == aOther.fFirst && fSecond == aOther.fSecond;
}


template<typename K, typename V>
bool SortablePair<K, V>::operator<( const SortablePair& aOther ) const noexcept
{
    return fFirst > aOther.fFirst;
}



template class SortablePair<size_t, int>;
std::ostream& operator<<( std::ostream& aOStream, const SortablePair<size_t, int>& aPair )
{
    aOStream << '(' << aPair.fFirst << ',' << aPair.fSecond << ')';
    return aOStream;
}

```